Analysis of Risky Borrowers

1 Introduction

This article builds upon the bad borrower model with graph features. While the previous work demonstrated how graph-based features can enhance supervised learning for predicting bad borrowers, this article focuses on leveraging graph-based representations to interpret and apply the results of such prediction models.

To summarize the key findings from the *bad borrower* prediction model: it successfully screened approximately 80% of the loans, leaving about 20% for further analyst review. When a loan is flagged as *bad* by the model, the next challenge is to interpret this outcome. The approach presented here addresses this by providing a framework for understanding the flagged loans.

Both the *bad borrower prediction* model and this work rely on the concept of a *neighborhood graph* as a central idea. However, while both use graph-based features, the method of featurization and the modeling approach differ. In this article, the focus shifts to an *unsupervised* approach, using a different featurization strategy to construct the neighborhood graph and interpret borrower profiles.

2 Methodology

1. We focus on borrowers identified as *risky*, specifically those located within the *risky neighborhood region*.

2. For interpretability, we featurize borrowers using their attributes in the raw data space (after cleaning), rather than the target-encoded features used in the bad borrower model. This approach ensures that similarities are based on demographic and business characteristics (e.g., state, business type), making the resulting clusters more meaningful for analysis.

3. The dataset contains both categorical (e.g., city, state) and numerical (e.g., loan term, interest rate) attributes. To measure similarity between borrowers with mixed data types, we use the *Gower Similarity* metric [2], which produces a pairwise similarity matrix.

4. Using this similarity matrix, we construct a *neighborhood graph* representing relationships among borrowers. We then apply *spectral clustering* to this graph. Spectral clustering is well-suited for data that may lie on complex, non-linear manifolds, as it leverages the structure of the neighborhood graph rather than relying solely on Euclidean

distances. For implementation details, see Section 3 and references [10, 9].

5. For each cluster, we compute the charge-off rate (i.e., the proportion of borrowers in the cluster who defaulted) using a simple group-by operation. This provides a cluster-level measure of risk.

6. Each risky borrower is assigned to a single cluster. When an analyst reviews a borrower flagged as risky by the prediction model, the risk profile of the borrower's cluster offers interpretable context for the decision.

7. The spectral clustering process also yields a low-dimensional *manifold* embedding of the borrowers, which can be visualized using techniques such as t-SNE [5] to further aid interpretation.

3 Clustering

This section outlines the main concepts and reasoning behind the use of *spectral clustering* in our analysis. For a comprehensive introduction to spectral clustering, refer to [9] and the tutorial by von Luxburg [10]. The construction of a *neighborhood graph* is central to our approach, as well as to the *bad borrower prediction* model. This technique is widely used to transform tabular (relational) data into a *homogeneous graph*[1] suitable for graph-based machine learning.

At the heart of spectral clustering lies the *Laplacian* matrix, which is derived from the neighborhood graph and encodes the local connectivity of each node. The eigenvalues and eigenvectors of the Laplacian provide critical information for clustering: specifically, the number of clusters is determined by identifying the *spectral gap*a significant jump in the sequence of the smallest eigenvalues. The corresponding eigenvectors are then used to embed the data in a lower-dimensional space where standard clustering algorithms, such as K-Means, can be applied. The reader interested in the connection between eigenvalues of the Laplacian and thier interpretaion, see [6].

In practice, the analyst inspects the first several (e.g., 50) eigenvalues of the Laplacian to locate the spectral gap, which indicates the optimal number of clusters. The number of eigenvectors used in the embedding typically matches the number of clusters. For implementation detailsincluding the construction of the Laplacian and the application of spectral clustering to this datasetsee this notebook.

[1] Homogeneous graphs are one of three common analysis patterns in business applications. See [8] for more details.

4 Results

The key results from applying *spectral clustering* to the risky neighborhood data is as follows. We start with the identification of the *spectral gap*. The plot of the first 50 or so eigenvalues is shown in the figure below. The location of the gap can be identified by calculating the differ-

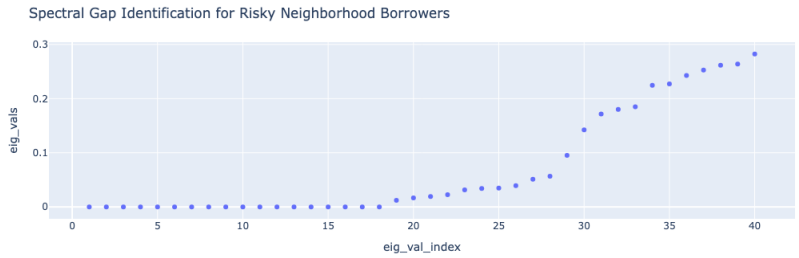

Figure 1: Spectral gap: Plot of the first 50 eigenvalues of the Laplacian matrix. The distinct jump indicates the optimal number of clusters.

ence between succesive eigenvalues and finding the maximum value of the gap. If the maximum gap occurs between values k and $k+1$, we use k as the optimal number of clusters. For the risky neighborhood data, k was 29. For references and a detailed explaination, please see [10].

Once the spectral gap is identified, we can generate the embedding and run $K-Means$ clustering with the optimal number of clusters on it. The embedding dimension is generally the same as the number of clusters. The embedding is the representation of the risky neighborhood in the *manifold*. We can then run *tsne* [5] to visualize the clusters, this is shown in 2

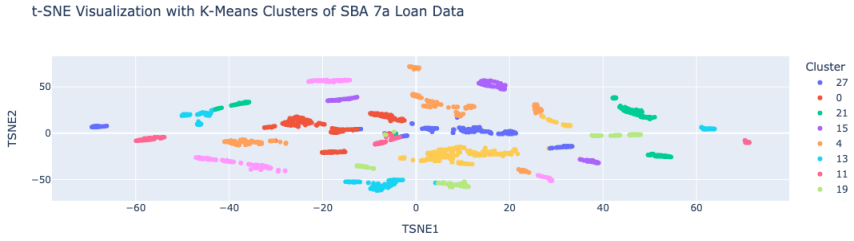

Figure 2: t-SNE visualization of risky neighborhood clusters.

We now examine key outcomes from the clustering analysis. One important aspect is the distribution of cluster sizes. Spectral clustering aims to produce partitions of comparable size, known as a *balanced cut*. As shown in Figure 4, the algorithm achieves a reasonable balance: with approximately $3{,}000$ borrowers in the risky neighborhood, there are no clusters that are disproportionately large or small.

Figure 3 displays the charge-off rates across these clusters. The results indicate that the *risky neighborhood* feature is highly informative- most clusters exhibit elevated charge-off rates. Thus, when a borrower is flagged as *bad* by the classifier, they are assigned to a cluster where the expected charge-off rate is consistently high. This tells the analyst reviewing the decision why the borrower is risky and how risky they are.

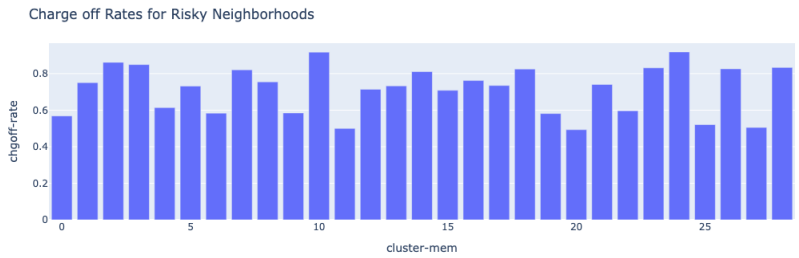

Figure 3: Charge off rates in the clusters from risky neighborhood data.

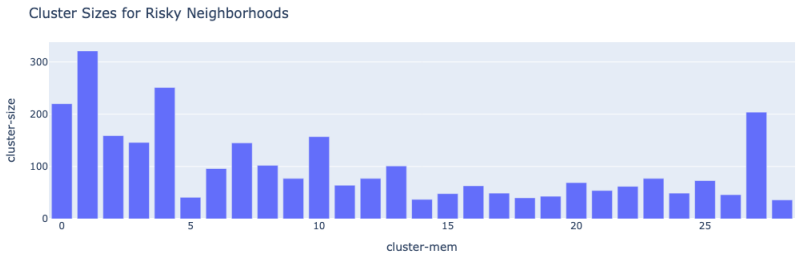

Figure 4: Cluster sizes for the risky neighborhood data.

The clustering can have useful applications. Instead of relying on an *average* loss estimate for lending to a risky borrower, assigning borrowers to specific clusters enables a more granular assessment of risk. This finer resolution is particularly valuable for structuring investment products, such as loan portfolios, where risk can be segmented into distinct tranches. By understanding the risk profile at the cluster level, analysts and investors can make more informed decisions and tailor products to different risk appetites. At a loan level, it is also possible to apply frameworks such as [7] to make cost-benefit based decisions.

5 Discussion and Insights

Both this example and the *bad borrower model with graph features* highlight the effectiveness of constructing a *homogeneous graph* from relational data

using an appropriate similarity measure. By carefully selecting a similarity or dissimilarity metric tailored to the dataset, one can generate a graph structure that captures meaningful relationships between examples. A central question in this process is: *How should two examples in the dataset be compared?* The answer to this guides the choice of similarity measure and, ultimately, the quality of the resulting graph-based analysis. A key step in graph-based machine learning is constructing a *vector* representation of the nodes or the graph itself, commonly referred to as an *embedding*. In this work, we employed a *spectral* embedding approach. The choice of embedding technique depends on factors such as the size of the graph and whether the model needs to generalize to previously unseen examples (i.e., whether the setting is transductive or inductive). The relation to graph idea focuses on building a graph from relational data, the process of embedding the graphtransforming it into a form suitable for downstream machine learning tasksrequires its own dedicated analysis. Please see [1], [3] for a detailed treatment of approaches to embedding a graph. The reader should be aware that there are other approaches to embedding a graph, such as *random walks* (see [4]). The spectral approach was used here because the size of the problem was amenable to this algorithm. There is also extensive literature on how to apply this algorithm in the wild. The approach of using one model to predict rare events and another to provide interpretability or context for positive predictions is a common pattern in applied analytics. This paradigm appears in domains such as fraud detection, healthcare interventions for rare diseases, and customer churn analysis. However, it is important to recognize that the choice of featuresand the method of featurizationcan vary significantly across applications. As demonstrated by the two SBA loan examples, effective featurization requires careful experimentation and validation in collaboration with domain experts to ensure that the features capture the relevant aspects of the problem.

References

[1] Ines Chami et al. "Machine Learning on Graphs: A Model and Comprehensive Taxonomy". In: *CoRR* abs/2005.03675 (2020). arXiv: 2005.03675. URL: https://arxiv.org/abs/2005.03675.

[2] John C Gower. "A general coefficient of similarity and some of its properties". In: *Biometrics* (1971), pp. 857–871.

[3] William L Hamilton. *Graph representation learning*. Morgan & Claypool Publishers, 2020.

[4] Jure Leskovic. *CS 224 Stanford Jure Leskovic Lecture on Random Walk Embeddings youtube.com*. https://www.youtube.com/watch?v=Xv0wRy66Big&t=961s. [Accessed 27-09-2025].

[5] Laurens van der Maaten and Geoffrey Hinton. "Visualizing data using t-SNE". In: *Journal of machine learning research* 9.Nov (2008), pp. 2579–2605.

[6] Anne Marsden. "Eigenvalues of the laplacian and their relationship to the connectedness of a graph". In: *University of Chicago, REU* (2013).

[7] Thomas Verbraken, Wouter Verbeke, and Bart Baesens. "A novel profit maximizing metric for measuring classification performance of customer churn prediction models". In: *IEEE transactions on knowledge and data engineering* 25.5 (2012), pp. 961–973.

[8] Ulrike Von Luxborg. *Statistical Machine Learning, Lecture 29-- youtube.com*. https://www.youtube.com/watch?v=8dxc3_00m3M&list=PL05umP7R6ij2XCvrRzLokX6EoHW index=32. [Accessed 01-10-2025].

[9] Ulrike Von Luxborg. *Statistical Machine Learning, Spectral Graph Theory Lect -- youtube.com*. https://www.youtube.com/watch?v=vCQmmeeSuuQ&list=PL05umP7R6ij2XCvrRzLokX6EoHWaGA2cC&index=38. [Accessed 01-10-2025].

[10] Ulrike Von Luxburg. "A tutorial on spectral clustering". In: *Statistics and computing* 17.4 (2007), pp. 395–416.